

Návod k projektu rozpoznávania hlasových príkazov

Odborná prax – aplikácie hlbokých neurónových sietí

Názov projektu: Aplikácie hlbokých neurónových sietí

Zameranie dokumentu: Tento text slúži ako praktický návod na prácu s repozitárom, prípravou datasetu, tréningom modelov a ich následným testovaním. Dokument je písaný tak, aby používateľ vedel jednotlivé kroky samostatne zopakovať a zároveň porozumel účelu pripravených skriptov.

Používané modely: V projekte sa porovnávajú tri architektúry, konkrétne model `raw-cnn`, model `raw-cnn-lstm` a model `spec-cnn`. Všetky skripty uvedené nižšie sú pripravené tak, aby medzi týmito modelmi bolo možné jednoducho prepínať pomocou parametra príkazového riadku.

1 Príprava prostredia

Táto kapitola opisuje úvodné kroky, ktoré je potrebné vykonať pred samotným spracovaním dát a tréningom modelov. Cieľom je pripraviť funkčné lokálne prostredie, v ktorom bude možné spúšťať skripty projektu reprodukovateľným spôsobom.

1.1 Klonovanie Git repozitára

Prvým krokom je získanie zdrojových súborov projektu. Repozitár je možné štandardne naklonovať pomocou systému Git, avšak z praktického hľadiska nie je nevyhnutné používať výlučne túto možnosť. Ak používateľ nechce pracovať s Gitom, môže si projekt stiahnuť aj vo forme archívu `.zip` a následne ho rozbaľiť do zvoleného pracovného priečinka.

Pri použití Git je typický postup nasledovný:

```
git clone https://github.com/mateusz13b/speech-recognition-playground.git
cd speech-recognition-playground
```

Ak je projekt stiahnutý ako `.zip`, je potrebné po rozbalení otvoriť terminál priamo v koreňovom adresári projektu. Práve z tohto umiestnenia sa následne spúšťajú všetky skripty uvedené v ďalších častiach návodu.

1.2 Inštalácia balíkov

Po získaní projektu je potrebné pripraviť Python prostredie a nainštalovať všetky závislosti. Bežný používateľ pri práci s projektom spravidla využíva vývojové prostredie, napríklad Visual Studio Code alebo PyCharm, pričom ďalšie kroky už stačí vykonávať priamo v termináli otvorenom v koreňovom adresári projektu.

Na inštaláciu balíkov možno použiť nasledujúce príkazy:

```
python -m pip install --upgrade pip
pip install -e .
```

Inštalácia cez `pip install -e .` načíta závislosti definované v súbore `pyproject.toml`. Medzi najdôležitejšie patria knižnice `torch`, `numpy`, `pandas`, `scikit-learn`, `matplotlib`, `soundfile` a `sounddevice`. V prípade tréningu na grafickej karte je zároveň potrebné mať nainštalovanú verziu PyTorch s podporou CUDA.

1.3 Script `prepare_data.py` a príprava datasetu

Skript `prepare_data.py` slúži na prípravu datasetu pre ďalšie fázy experimentu. Vyhľadá archív alebo rozbalený dataset, v prípade potreby vykoná extrakciu, vytvorí rozdelenie dát na tréningovú, validačnú a testovaciu časť a uloží prehľadové súbory, s ktorými následne pracujú tréningové a evaluačné skripty.

Projekt je navrhnutý pre dataset Google Speech Commands. Je potrebné ho umiestniť do priečinka `data/raw/`, a to buď vo forme archívu `speech_commands_v0.02.tar.gz`, alebo ako už rozbalený adresár `speech_commands_v0.02`. Zároveň je potrebné uviesť, že dataset bude poskytnutý iba tým študentom, ktorým ho sprístupní vyučujúci na základe vlastného rozhodnutia. Autor tohto návodu k datasetu nemá priamy prístup a jeho distribúciu nezabezpečuje.

Samotná príprava dát sa vykoná príkazom:

```
python scripts/prepare_data.py
```

Voliteľne možno zadať aj vlastné cesty pomocou parametrov `-raw-root` a `-output-dir`. Po úspešnom spracovaní vzniknú najmä súbory `data/processed/dataset_index.csv` a `data/processed/summary.json`. Tieto výstupy predstavujú základnú indexáciu datasetu a ich existencia je predpokladom pre nasledujúci tréning modelov.

2 Trénovanie modelu

Táto kapitola sa venuje konfigurácii a spusteniu tréningu. Používateľ si môže zvoliť konkrétnu architektúru, upraviť jej parametre v zdrojovom kóde a následne spustiť tréning s vybranými nastaveniami pomocou pripraveného skriptu.

2.1 Nastavenie štruktúry jednotlivých sietí v `models.py`

Definície modelov sa nachádzajú v súbore `src/iop/models.py`. V projekte sú implementované tri architektúry: `Conv1DClassifier` pre model `raw-cnn`, `Conv1DLSTMClassifier` pre model `raw-cnn-lstm` a `SpectrogramCNN` pre model `spec-cnn`. Tento súbor je určený na experimentovanie so štruktúrou neurónových sietí.

Používateľ môže meniť napríklad počet konvolučných vrstiev, počet kanálov, veľkosti jadier, pooling, parametre LSTM vrstvy, dropout alebo klasifikačnú hlavu. Pri každej významnej zmene architektúry je potrebné počítať s tým, že staršie uložené váhy alebo checkpointy už nemusia byť kompatibilné s novou definíciou modelu. V takom prípade je nevyhnutné vykonať nový tréning a vytvoriť nové výstupné súbory.

Táto podkapitola je dôležitá najmä pri porovnávaní vplyvu architektonických úprav na presnosť klasifikácie a stabilitu tréningu. Odporúča sa upravovať iba jeden aspekt modelu naraz, aby bolo možné výsledky interpretovať korektne.

2.2 Spustenie a parametre tréningu

Na spustenie tréningu slúži skript `scripts/train.py`. Základný spôsob použitia je voľba názvu modelu a počtu epôch, pričom ďalšie parametre možno pridávať podľa potreby experimentu.

Príklad základného spustenia je:

```
python scripts/train.py --model-name raw-cnn --epochs 10 --batch-size 32
```

Podporované hodnoty parametra `-model-name` sú `raw-cnn`, `raw-cnn-lstm` a `spec-cnn`. Skript okrem toho podporuje aj parametre `-learning-rate`, `-weight-decay`, `-num-workers`, `-seed`, `-device`, `-max-train-samples` a `-max-val-samples`. Parametrom `-device` je možné explicitne zvoliť `cpu`, `cuda` alebo ponechať predvolenú hodnotu `auto`.

Pre rýchle experimenty na malej časti datasetu je vhodné obmedziť počet vzoriek, napríklad:

```
python scripts/train.py --model-name raw-cnn --epochs 1  
--max-train-samples 8 --max-val-samples 4
```

Takýto režim je užitočný pri kontrole správnosti pipeline, pri ladení parametrov alebo pri overení, či modifikovaný model beží bez chyby ešte pred dlhším tréningom.

2.3 Evaluácia modelu a ukladanie váh

Počas tréningu sa priebežne ukladajú dôležité výstupy, ktoré umožňujú analyzovať správanie modelu a neskôr ho znovu použiť. Projekt ukladá jednak kompletný checkpoint modelu, jednak samotné váhy, ako aj metriky a grafické výstupy.

Na samostatnú evaluáciu natrénovaného modelu na testovacej množine slúži skript `scripts/evaluate.py`. Príklad použitia je:

```
python scripts/evaluate.py --model-name raw-cnn
```

Skript podporuje aj parametre `-batch-size`, `-device` a `-max-test-samples`. Medzi ukladané súbory patria najmä `results/checkpoints/<model-name>.pt`, `results/weights/<model-name>_weights.pt` a súbory v priečinku `results/metrics/`, ktoré obsahujú históriu tréningu, testovacie metriky, klasifikačný report a konfúznú maticu. Tieto výstupy sú nevyhnutné na vecné porovnanie modelov a na dokumentovanie výsledkov experimentu.

3 Testovanie modelu

Po natrénovaní a vyhodnotení modelu nasleduje praktická fáza testovania. V nej sa overuje, či uložené váhy dokážu správne klasifikovať nové zvukové ukážky pripravené mimo tréningovej množiny.

3.1 Príprava zvukových súborov

Na testovanie je potrebné pripraviť vlastné zvukové nahrávky. Používateľ si musí audio nahráť samostatne v externom nástroji a následne ho skonvertovať do formátu WAV so vzorkovacou frekvenciou 16 kHz a s jedným kanálom, teda mono. Odporúčaná dĺžka jednej ukážky je približne jedna sekunda.

Táto požiadavka je dôležitá preto, že predspracovanie v projekte očakáva práve takýto vstupný formát. Ak nahrávka nebude mať požadované parametre, môže dôjsť k nesprávnej interpretácii signálu alebo k zhoršeniu presnosti klasifikácie. Na jednoduchú konverziu možno odporučiť aj službu `cloudconvert.com`, kde je možné explicitne nastaviť výstupný formát WAV, jeden kanál typu mono a vzorkovaciu frekvenciu 16 kHz. Z praktického hľadiska je vhodné pripraviť si niekoľko krátkych ukážok s rôznymi príkazmi a uložiť ich napríklad do samostatného priečinka `test/`.

3.2 Jednoduchý test váh

Na jednoduché overenie funkčnosti natrénovaných váh slúži skript `scripts/predict_file.py`. Tento skript načíta pripravený súbor `.wav`, vyhľadá uložené váhy zvoleného modelu a vypíše predikovaný príkaz spolu s hodnotou dôvery a s prehľadom najpravdepodobnejších tried.

Príklad použitia je:

```
python scripts/predict_file.py --model-name raw-cnn --wav test/stop.wav
```

Výstup skriptu obsahuje cestu k súboru, názov modelu, umiestnenie použitých váh, výslednú triedu a zoznam najpravdepodobnejších predikcií. V projekte sa nachádza aj skript `scripts/demo_mic.py`, avšak v aktuálnom stave ešte neobsahuje implementované spracovanie mikrofónu. Z tohto dôvodu je odporúčaný praktický postup taký, že sa nahrávka vytvorí externe, upraví sa na požadovaný formát a následne sa vyhodnotí pomocou skriptu `predict_file.py`.

4 Experimentálne výsledky a porovnanie modelov

Táto kapitola sumarizuje experimentálnu časť projektu. Jej cieľom je zhrnúť vlastnosti použitého datasetu, presne opísať implementované architektúry, uviesť parametre tréningu a následne porovnať dosiahnuté výsledky pomocou číselných metrík aj grafických výstupov.

4.1 Dataset a rozdelenie na train, validation a test

Projekt využíva dataset Google Speech Commands, ktorý obsahuje krátke hlasové povely nahraté rôznymi hovorcami. V tejto práci sa používa osem cieľových príkazov, konkrétne `go`, `stop`, `left`, `right`, `on`, `off`, `up` a `down`. Popri nich projekt pracuje aj s triedami `unknown` a `background`, ktoré pomáhajú modelu lepšie rozlišovať relevantné a nerelevantné vstupy.

Rozdelenie datasetu sa vytvára skriptom `prepare_data.py` v režime `speaker-aware`. To znamená, že nahrávky jedného hovoriaceho nie sú rozdelené medzi tréningovú, validačnú a testovaciu časť náhodne po súboroch, ale s ohľadom na identitu hovoriaceho. Takýto postup znižuje riziko nadhodnotenia výsledkov a lepšie simuluje reálne použitie modelu na nových hlasoch. Použité rozdelenie má pomer približne 80 % pre tréning, 10 % pre validáciu a 10 % pre testovanie.

4.2 Použité architektúry neurónových sietí

V súbore `src/iop/models.py` sú implementované tri modelové architektúry, ktoré predstavujú tri odlišné prístupy k spracovaniu zvukového signálu. Každá z nich pracuje s rovnakou klasifikačnou úlohou, no odlišuje sa spôsobom reprezentácie vstupu aj vnútornou štruktúrou siete.

Prvá architektúra, označená ako štruktúra 1 a implementovaná triedou `Conv1DClassifier`, pracuje priamo so surovou časovou reprezentáciou zvuku. Model začína konvolučnou vrstvou `Conv1d(1, 32, kernel_size=80, stride=16)`, po ktorej nasledujú normalizácia, aktivačná funkcia `ReLU` a `max-pooling`. Následne obsahuje ďalšie tri jednorozmerné konvolučné bloky, pričom počet kanálov rastie z 32 na 64. Po adaptívnom priemerovaní na dĺžku jedna nasleduje klasifikačná časť s vrstvami `Linear(64, 64)`, `ReLU`, `dropout` s hodnotou 0.2 a finálna lineárna vrstva do počtu tried.

Druhá architektúra, označená ako štruktúra 2 a implementovaná triedou `Conv1DLSTMClassifier`, kombinuje jednorozmerné konvolúcie a rekurentnú zložku LSTM. Predspracovacia časť obsahuje dve vrstvy `Conv1d` s `base_channels=64`, medzi ktorými sú opäť normalizačné vrstvy, `ReLU` a `max-pooling`. Výstup z konvolučného frontendu sa následne transponuje a spracuje dvojvrstvovou obojsmernou LSTM s parametrami `hidden_size=128`, `num_layers=2` a `dropout=0.25`. Klasifikačná hlava pozostáva z vrstvy `Linear(256, 128)`, aktivačnej funkcie `ReLU`, `dropout=0.3` a finálnej lineárnej vrstvy do priestoru tried.

Tretia architektúra, označená ako štruktúra 3 a implementovaná triedou `SpectrogramCNN`, používa ako vstup spektrogram namiesto surového priebehu signálu. Sieť pozostáva zo štyroch dvojrozmerných konvolučných blokov s počtami kanálov 32, 64, 128 a 128. Prvé tri bloky obsahujú aj `max-pooling` po každej konvolúcii, zatiaľ čo záverečný blok je ukončený adaptívnym priemerovaním na rozmer 1×1 . Po sploštení nasleduje lineárna vrstva `Linear(128, 128)`, `ReLU`, `dropout` s hodnotou 0.3 a výstupná lineárna vrstva. Táto architektúra využíva priestorovo-frekvenčnú reprezentáciu signálu a je vhodná najmä vtedy, keď sa chcú zachytiť lokálne vzory v spektre.

4.3 Parametre tréningu a spustenia

Tréning všetkých troch modelov sa spúšťa skriptom `scripts/train.py`, pričom medzi modelmi sa prepínalo pomocou parametra `-model-name`. Zo súborov s históriou tréningu vyplýva, že pre všetky tri modely bolo vykonaných 50 epôch. Základné spustenie tréningu má tvar:

```
python scripts/train.py --model-name raw-cnn --epochs 50
```

V analogickej forme sa spúšťali aj modely `raw-cnn-lstm` a `spec-cnn`. Skript podporuje aj ďalšie parametre, najmä `-batch-size`, `-learning-rate`, `-weight-decay`, `-num-workers`, `-seed` a `-device`. Keďže aktuálna implementácia tréningového skriptu má predvolené hodnoty `batch-size=64`, `learning-rate=0.001`, `weight-decay=0.0`, `num-workers=0`, `seed=42` a `device=auto`, tieto hodnoty predstavujú základnú referenčnú konfiguráciu, pokiaľ pri spustení neboli explicitne prepísané iným argumentom.

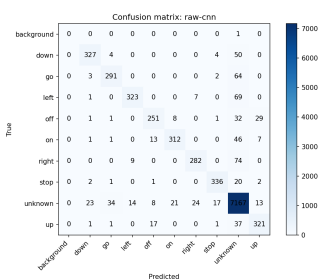
Z praktického hľadiska je dôležité uviesť, že modely sa po tréningu ukladajú ako checkpointy aj ako samostatné váhy. Vďaka tomu možno neskôr vykonať evaluáciu bez nutnosti opakovať celý tréningový proces. Tento postup je vhodný najmä pri porovnávaní viacerých architektúr na rovnakom datasete.

4.4 Evaluácia modelu a grafické výstupy

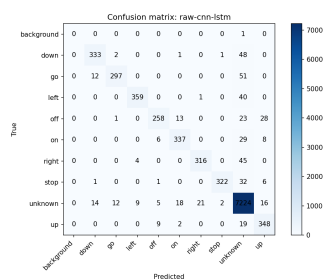
Vyhodnotenie natrénovaných modelov sa vykonáva skriptom `scripts/evaluate.py`. Základný spôsob použitia je:

```
python scripts/evaluate.py --model-name raw-cnn
```

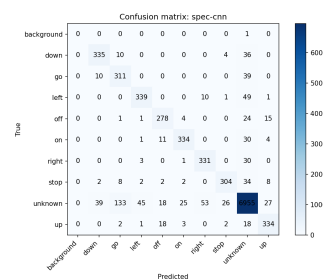
Analogicky možno použiť aj hodnoty `raw-cnn-lstm` a `spec-cnn`. Skript načíta príslušný model, spustí evaluáciu na testovacej časti datasetu a uloží číselné metriky aj grafické výstupy do priečinkov `results/metrics/` a `results/plots/`. Pre každú architektúru sú k dispozícii najmä graf konfúznej matice, graf priebehu straty v závislosti od epochy a graf priebehu presnosti modelu.



(a) Štruktúra 1: raw-cnn

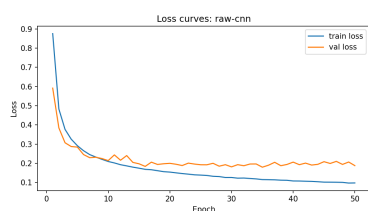


(b) Štruktúra 2: raw-cnn-lstm

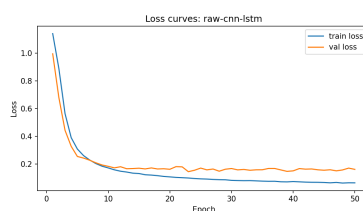


(c) Štruktúra 3: spec-cnn

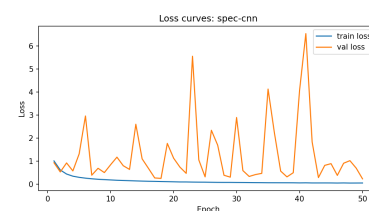
Obr. 1: Konfúzne matice troch použitých architektúr.



(a) Štruktúra 1: raw-cnn

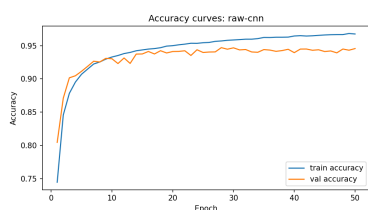


(b) Štruktúra 2: raw-cnn-lstm

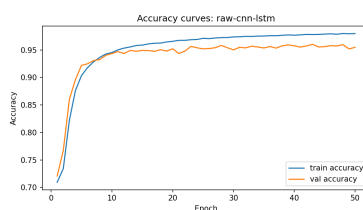


(c) Štruktúra 3: spec-cnn

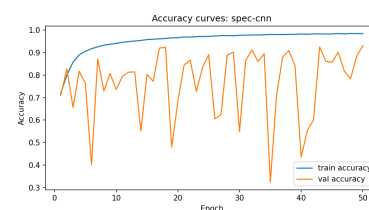
Obr. 2: Priebeh tréningovej straty v závislosti od epochy pre tri použité architekúry.



(a) Štruktúra 1: raw-cnn



(b) Štruktúra 2: raw-cnn-lstm



(c) Štruktúra 3: spec-cnn

Obr. 3: Priebeh presnosti modelu pre tri použité architekúry.

Z grafických výstupov je viditeľné, že model raw-cnn-lstm dosahuje najstabilnejší priebeh učenia a zároveň aj najlepšie oddelenie tried v konfúznej matici. Model raw-cnn je jednoduchší a rýchlo použiteľný, no v porovnaní s ostatnými architekturami dosahuje slabšie výsledky najmä pri zložitejšom rozlíšení hlasových príkazov. Model spec-cnn pracuje so spektrogramom a poskytuje solídny kompromis medzi presnosťou a robustnosťou, hoci jeho validačný priebeh je menej stabilný než pri architektúre s LSTM.

4.5 Porovnanie metrík jednotlivých modelov

V tabuľke nižšie sú zhrnuté hlavné výsledky troch architektúr. Hodnoty accuracy sú prevzaté z testovacích metrík a hodnoty precision a recall zodpovedajú makropriemeru cez všetky triedy, aby porovnanie nebolo skreslené dominanciou triedy `unknown`.

Tabuľka 1: Porovnanie metrík troch použitých architektúr na testovacej množine.

Štruktúra siete	Accuracy	Precision	Recall
1 – raw-cnn	0.8526	0.6806	0.6143
2 – raw-cnn-lstm	0.9532	0.8420	0.7936
3 – spec-cnn	0.9266	0.7721	0.7901

Na základe týchto výsledkov možno konštatovať, že najvyššiu presnosť aj najvyššiu hodnotu makropriemernej precision dosiahla architektúra `raw-cnn-lstm`. Model `spec-cnn` dosiahol veľmi podobnú hodnotu recall, čo naznačuje dobrú schopnosť zachytiť väčšinu cieľových tried, avšak celkovo zaostal za kombináciou konvolučnej a rekurentnej zložky. Najjednoduchší model `raw-cnn` slúži ako vhodná referenčná základňa, no v tomto experimente nedosiahol rovnakú kvalitu ako zvyšné dve architektúry.